Guia do Desenvolvedor — JTe Monitor / Ativa.LAW

Este documento complementa o `README.md` (focado em deploy/infra) com o que um desenvolvedor **novo no projeto** precisa saber para entender o código, dar suporte, corrigir bugs e implementar novas features com segurança.

> Convenção do projeto: nomes de variáveis, funções, rotas e mensagens ao usuário são em **português**. Siga esse padrão em código novo.

Índice

1. O que o produto faz
2. Arquitetura em um parágrafo
3. Stack tecnológica
4. Mapa do repositório
5. Setup local passo a passo
6. Backend — como o NestJS está organizado
7. Autenticação e tiers
8. Banco de dados e migrations
9. O worker/scraper
10. Integrações externas (Snov.io, Gemini, Nanonets)
11. Frontend — como o Next.js está organizado
12. Fluxo de dados ponta a ponta
13. Como adicionar uma feature nova (receita)
14. Dívida técnica e pontos de atenção conhecidos
15. Onde procurar cada coisa (índice rápido)

---

1. O que o produto faz

O JTe Monitor (marca comercial **Ativa.LAW**) é um SaaS jurídico que raspa a pauta pública do JTe (sistema de audiências do TRT-2 São Paulo, zonas Central, Norte e Oeste) e identifica **processos trabalhistas em que a reclamada ainda não tem advogado constituído no polo passivo** — ou seja, leads de captação de clientes para escritórios de advocacia.

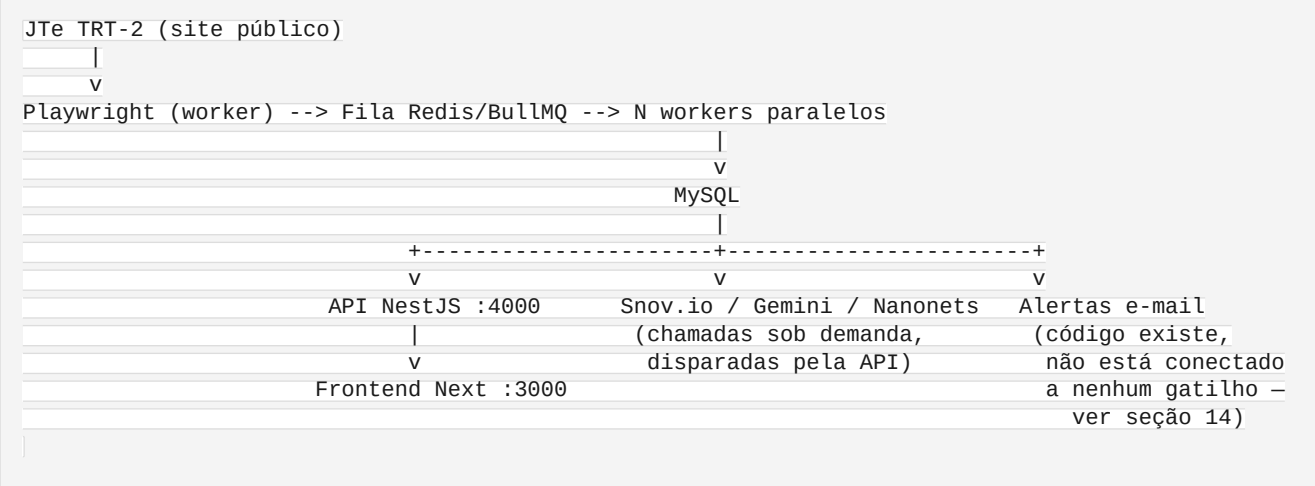
A partir desses processos, o produto foi crescendo com módulos correlatos:

- **Processos** — lista principal de audiências sem advogado no polo passivo.

- **Contatos** — e-mails/CNPJ das reclamadas extraídos de processos.
- **Contatos Snov** — enriquecimento desses contatos via Snov.io (achar pessoas/e-mails da empresa reclamada, não só o e-mail genérico).
- **Contratos** — upload de contrato e análise de risco via IA (Gemini).
- **Conversões** — OCR de documento para texto/markdown via Nanonets.
- **Analytics** — painel interno (`admin`) de uso da plataforma.

2. Arquitetura em um parágrafo

Um **worker** headless (Playwright) raspa o site do JTe e grava processos no **MySQL**; ele usa **Redis/BullMQ** só para paralelizar (fila de jobs vara×data, N browsers consumindo em paralelo). Uma **API NestJS** (porta 4000) expõe esses dados via REST autenticado por **JWT** (access token curto + refresh token em cookie httpOnly). Um **frontend Next.js** (porta 3000) consome a API e mostra tudo em abas dentro de um dashboard único. Tudo roda atrás de **Nginx** fazendo proxy reverso + TLS (Let's Encrypt) em produção, ou via **Docker Compose** (ver `docker-compose.yml`).



3. Stack tecnológica

Camada	Tecnologia	Onde
API	NestJS 10 + TypeScript	`backend/src/`
ORM/DB driver	`mysql2/promise` puro (sem ORM — SQL direto)	`backend/src/common/db.service.ts`
Auth	JWT (`@nestjs/jwt` + `passport-jwt`) + bcrypt	`backend/src/auth/`
Fila	BullMQ + ioredis	`backend/worker/index.js`
Scraper	Playwright (Chromium headless)	`backend/worker/index.js`

Camada	Tecnologia	Onde
Frontend	Next.js 14 (App Router) + React 18, sem CSS framework (estilos inline)	`frontend/src/app/`
Banco	MySQL 8, `utf8mb4`	`database/*.sql`
E-mail	Nodemailer (SMTP, ex.: Gmail App Password)	`backend/src/alertas/`
IA análise de contrato	Google Gemini (`@google/generative-ai`)	`backend/src/contratos/gemini.service.ts`
OCR	Nanonets Extraction API	`backend/src/conversoes/nanonets.service.`
Enriquecimento de contatos	Snov.io API	`backend/src/contatos/snov-live.service.ts`

Não há suíte de testes automatizados no projeto (nenhum `*.spec.ts` ou `*.test.js`) e não há linter configurado. Validação de tipos é feita pelo `tsc` do NestJS (`npm run build`) — rode-o antes de dar push em mudanças no backend.

4. Mapa do repositório

```
jte-monitor/
+-- database/                      Migrations SQL, numeradas e rodadas em ordem
|   +-- 000_init.sql               Cria o banco + usuários MySQL (roda manual, 1x, como root)
|   +-- 001_users.sql              users, refresh_tokens
|   +-- 002_processos.sql          processos_sem_polo_passivo, execucoes
|   +-- 003_alertas.sql            alertas, notificacoes
|   +-- 004_contatos.sql           contatos_reclamadas
+-- migrate.js                     Roda todas as migrations 001+ em ordem (não 000)
+-- backend/
|   +-- src/
|   |   +-- main.ts                Bootstrap NestJS, CORS, cookie-parser, ValidationPipe
|   |   +-- app.module.ts          Módulo raiz — TODO controller/provider é registrado aqui
|   |   +-- auth/                  Login, registro, refresh, logout, estratégia JWT
|   |   +-- users/                  CRUD básico de usuário + tier
|   |   +-- processos/             GET /api/processos (lista principal)
|   |   +-- alertas/                CRUD de alertas + envio de e-mail (premium)
|   |   +-- contatos/                Contatos extraídos de processos + enriquecimento Snov.io
|   |   +-- contratos/             Upload de contrato + análise de risco (Gemini)
|   |   +-- conversoes/            OCR documento -> texto (Nanonets)
|   |   +-- admin/                  Analytics interno (tier admin)
|   |   +-- common/
|   |       +-- db.service.ts       Pool MySQL singleton, injetado em todo lugar
|   |       +-- guards/index.ts    JwtAuthGuard, TierGuard, @Tiers(), @CurrentUser()
|   +-- worker/index.js            Scraper Playwright + fila BullMQ (roda fora do NestJS)
+-- contatos/
|   +-- snov.js                    Script standalone (não parte da API) para enriquecer
|                                   contatos_reclamadas em lote via Snov.io — roda manual/cron
+-- frontend/
|   +-- src/
|       +-- lib/auth.js            AuthContext React: login/logout/refresh/authFetch
|       +-- app/
|           +-- layout.js           Envolve tudo em <AuthProvider>
|           +-- page.js             "/" redireciona para "/login"
|           +-- login/page.js       Tela de login + cadastro
|           +-- dashboard/page.js Dashboard com abas (Processos/Contatos/Snov/Contratos/…)
```

```
|           +-- upgrade/page.js   Comparação de planos free × premium
+-- docker-compose.yml           Sobe jte-api e jte-frontend (não sobe MySQL/Redis)
```

5. Setup local passo a passo

Pré-requisitos: Node 20, MySQL 8 rodando, Redis rodando (só necessário se for testar o worker).

```
# 1. Banco
mysql -u root -p < database/000_init.sql      # edite as senhas TROQUE_AQUI antes
cp backend/.env.example backend/.env          # preencha DB_PASS, JWT_SECRET, SMTP_*
node migrate.js                               # roda a partir da raiz do repo

# 2. Backend
cd backend
npm install
npm run dev           # ts-node-dev com hot-reload, porta 4000

# 3. Frontend (outro terminal)
cd frontend
cp .env.local.example .env.local     # NEXT_PUBLIC_API_URL=http://localhost:4000
npm install
npm run dev           # porta 3000
```

Acesse `http://localhost:3000`, crie uma conta (fica com tier `free` por padrão) e faça login. Para testar features premium, promova o usuário direto no banco:

```
UPDATE users SET tier = 'premium' WHERE email = 'voce@exemplo.com';
```

Para ver dados reais de processos sem rodar o scraper completo (que demora 30-40 min), rode o worker limitado:

```
cd backend
CONCURRENCY=1 MESES=1 VARAS=2 node worker/index.js
```

Detalhes de deploy em produção (systemd, Nginx, HTTPS, cron) estão no `README.md`, seções 6 a 10 — este guia não repete esse conteúdo.

6. Backend — como o NestJS está organizado

Não há service layer em todo módulo — o padrão do projeto é **controller enxuto com SQL inline via `DbService`**, sem repositório/ORM. Alguns módulos têm um `Service` separado (quando há lógica não-trivial: `AlertasService`, `SnovLiveService`, `GeminiService`, `NanonetsService`, `UsersService`), outros colocam tudo direto no controller (`ProcessosController`, `ContatosController`). Siga o padrão do módulo que você está tocando.

Tudo é registrado manualmente em `app.module.ts` — **não existem sub-módulos NestJS** (`@Module` por feature). Ao criar um controller/service novo, você precisa importá-lo e listá-lo em `controllers` / `providers` nesse arquivo raiz, senão o NestJS nunca vai instanciá-lo.

`DbService` (`backend/src/common/db.service.ts`) expõe só um método, `query<T>(sql, params)`, que devolve `[rows, fields]` (formato do `mysql2/promise`). Não há transações usadas em lugar nenhum do código atual —

se precisar de uma operação atômica multi-tabela, você vai precisar implementar isso (o `Pool` do `mysql2` suporta `getConnection()` + `beginTransaction()`).

Padrão de resposta paginada usado em quase todo `GET` de listagem (`processos`, `contatos`, `contatos-snov`): `{ rows, total, pagina, totalPaginas, ...filtrosDisponiveis }`. Mantenha esse formato em endpoints novos de listagem para o frontend reaproveitar os mesmos componentes.

7. Autenticação e tiers

- **Access token**: JWT assinado com `JWT_SECRET`, expira em `JWT_ACCESS_EXPIRES` (padrão 15 min), enviado no header `Authorization: Bearer`. Payload: `{ sub: userId, email, tier }`.
- **Refresh token**: string aleatória de 64 bytes, guardada na tabela `refresh_tokens`, entregue em cookie `httpOnly` chamado `jte_refresh` (escopo `path: /api/auth`). Rotativo: cada `POST /api/auth/refresh` revoga o antigo e emite um novo. Só os 5 mais recentes por usuário são mantidos (`auth.service.ts`, método `_tokens`).
- O frontend nunca guarda o refresh token em JS — ele vive só no cookie. O access token fica em memória (`useState` do `AuthContext`, não `localStorage`), e é perdido a cada reload de página; por isso `AuthProvider` chama `refresh()` automaticamente no mount (`frontend/src/lib/auth.js`).
- `authFetch()` no frontend é o wrapper que toda chamada à API deveria usar: ele injeta o Bearer token e, se receber 401, tenta um `refresh()` e repete a chamada uma vez antes de desistir.

Tiers: `free`, `premium`, `admin` (coluna `users.tier`, enum no MySQL). Autorização por tier é feita com o guard `TierGuard` + decorator `@Tiers(...)` (`backend/src/common/guards/index.ts`). Exemplo de uso:

```
@UseGuards(JwtAuthGuard, TierGuard)
@Tiers("premium", "admin")
export class AlertasController { ... }
```

Sem `@Tiers()`, `TierGuard` deixa passar qualquer tier autenticado — o guard só bloqueia quando há metadata de tiers exigidos na rota/classe.

Promover um usuário a admin/premium hoje é feito só via SQL direto (`UPDATE users SET tier=...`) — não existe endpoint de admin para isso.

8. Banco de dados e migrations

Migrations ficam em `database/*.sql`, numeradas (`001_`, `002_`, ...). Rodar `node migrate.js` (a partir da raiz) executa em ordem **todo arquivo `.sql` da pasta `database/` exceto `000_init.sql`** (que é rodado manualmente, uma vez, como root — cria o banco e os usuários MySQL).

`migrate.js` não é idempotente-consciente por versão (não há tabela de controle tipo `schema_migrations`) — ele reexecuta tudo toda vez que é chamado. Isso funciona porque toda migration usa `CREATE TABLE IF NOT EXISTS`. **Ao escrever uma migration nova, sempre use `IF NOT EXISTS` e evite `ALTER TABLE` destrutivo** — se precisar alterar uma tabela existente, prefira criar a migration com `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` (MySQL 8 suporta) e teste em uma cópia do banco antes.

Tabelas principais:

Tabela	Migration	Descrição
`users`	001	Conta do usuário, `tier` enum
`refresh_tokens`	001	Refresh tokens rotativos
`processos_sem_polo_passivo`	002	Resultado do scraper
`execucoes`	002	Log de cada run do worker
`alertas`	003	Filtros salvos (premium)
`notificacoes`	003	Histórico de e-mails enviados (evita reenvio)
`contatos_reclamadas`	004	E-mail/CNPJ da reclamada extraído do processo

Há duas tabelas usadas pelo código que **não têm migration correspondente** no repositório — ver seção 14 (`contatos_snov`, `sessoes_acesso`). Se você está montando um ambiente novo do zero e algo falhar com "table doesn't exist" nessas duas, é por isso: crie a tabela manualmente inspecionando as colunas usadas no código (`snov-live.service.ts` para `contatos_snov`; `auth.controller.ts` / `analytics.controller.ts` para `sessoes_acesso`) e, idealmente, formalize a migration que falta.

9. O worker/scraper

`backend/worker/index.js` é um script Node standalone — **não passa pelo NestJS**, roda com `node worker/index.js` (via cron, `screen` / `tmux`, ou manual). Ele:

1. Abre um browser Playwright, navega até `jte.csjt.jus.br`, seleciona TRT-2 SP / zonas Central-Norte-Oeste, e lista todas as varas (unidades) disponíveis (`listarVaras`).
2. Gera uma lista de datas úteis (seg-sex) para os próximos `MESES` meses.
3. Enfileira um job BullMQ por combinação vara×data (`--only-queue` só faz isso).
4. `CONCURRENCY` workers consomem a fila em paralelo, cada um com seu próprio browser Chromium. Cada job (`processarJob`) navega até a vara/data, extrai a lista de processos da pauta, **abre o detalhe de cada processo para verificar se há advogado no polo passivo** (`semAdvogado` — isso é caro: 1 navegação extra por processo) e só salva os que não têm.
5. Grava no MySQL com `INSERT ... ON DUPLICATE KEY UPDATE` — reexecutar o worker é seguro, nunca duplica (chave única `vara+dataISO+numeroProcesso`).

Pontos importantes para quem for mexer aqui:

- O scraping depende de seletores de UI Angular/Ionic frágeis (`mat-select`, `ion-list`, XPath de botões de navegação de data). Se o JTe atualizar a UI, isso quebra silenciosamente — sempre teste com `CONCURRENCY=1 MESES=1 VARAS=1` e `DEBUG_HTML=1` (salva o HTML da página de detalhe em `backend/worker/output/`) antes de rodar a carga completa.

- `dataJaPassou()` filtra jobs cuja data já passou (pode acontecer se a fila ficar dias parada) — não remova sem entender o motivo.
- O worker lê `.env` de um caminho **absoluto hardcoded**: `require("dotenv").config({ path: "/root/jte-monitor/backend/.env" })` (linha 12). Isso só funciona se o repositório estiver clonado exatamente nesse caminho no servidor. Se o deploy usa outro caminho (o README usa `/opt/jte-monitor`), **ajuste essa linha** ou rode com `dotenv_config_path` — do contrário o worker sobe sem variáveis de ambiente e falha ao conectar no banco/Redis.
- `worker/index.js.bak` é um backup manual deixado no repo — não é usado por nada, pode ser ignorado (ou removido se confirmar que não é mais precisa).

10. Integrações externas (Snov.io, Gemini, Nanonets)

Todas seguem o mesmo padrão: token/chave de API vem do `.env`, chamada via `fetch` nativo (Node 20+ tem fetch global), sem SDK oficial.

- **Snov.io** (`SNOV_CLIENT_ID` / `SNOV_CLIENT_SECRET`) — busca prospects e e-mails de um domínio. Fluxo é assíncrono no lado da Snov.io: você chama um `/start`, recebe um `task_hash`, e faz polling em `/result/:hash` até `status: "completed"` (helper `pollResult`, até 15 tentativas × 4s). Usado em dois lugares diferentes que fazem a mesma coisa:
- `backend/src/contatos/snov-live.service.ts` — chamado **on-demand pela API** quando alguém filtra `ContatosSnovController` por um domínio exato ainda não cacheado (busca ao vivo, salva em `contatos_snov`, e a mesma request já devolve o resultado).
- `contatos/snov.js` — script **standalone em lote** (não integrado à API, tem seu próprio `.env` em `contatos/.env`), para enriquecer todos os domínios de `contatos_reclamadas` de uma vez. Se for alterar a lógica de enriquecimento, os dois arquivos duplicam a mesma lógica — considere unificar ou pelo menos replicar a mudança nos dois.
- **Gemini** (`GEMINI_API_KEY`, `GEMINI_MODEL`) — análise de risco de contrato, multimodal (recebe o PDF direto em base64, sem OCR prévio). Só aceita `application/pdf` e `text/plain` hoje (`MIME_PERMITIDOS` em `gemini.service.ts`) — apesar da mensagem de erro mencionar DOC/DOCX, esses formatos **não são aceitos** (precisaria de conversão prévia, fora de escopo atual).
- **Nanonets** (`NANONETS_API_TOKEN`) — OCR de documento para markdown, fluxo em 2 passos (upload -> parse síncrono). Endpoint legado documentado em comentário no topo do arquivo não existe mais — se for revisar essa integração, confira a doc oficial linkada no comentário, a API muda.

Todas essas três chamam serviços pagos por uso — tenha cuidado ao testar em loop/dev para não gerar custo desnecessário.

11. Frontend — como o Next.js está organizado

App Router do Next.js 14, sem TypeScript, sem framework de CSS (todo estilo é inline via objetos `style={{}}`) e sem biblioteca de componentes — cada página define seus próprios componentes de UI reutilizáveis no topo do arquivo (`FiltroInput`, `BtnPrimario`, `StatCard`, etc.).

- **`lib/auth.js`** — `AuthProvider` / `useAuth()`. Único lugar que fala com a API para autenticação. Expõe `authFetch`, que é o que toda chamada autenticada no dashboard usa (ver seção 7).

- `app/layout.js` — layout raiz, só envolve tudo em `<AuthProvider>`.
- `app/page.js` — rota `/` redireciona para `/login` (não para `/dashboard`; se você chegar aqui vindo do README, note que a versão atual do código difere do que está documentado lá).
- `app/login/page.js` — formulário único de login + cadastro (alterna entre os dois modos).
- `app/dashboard/page.js` — **arquivo único de quase 1000 linhas**, é o coração do frontend. Estrutura: um componente de página que renderiza uma barra de abas e troca qual `Aba*` está montada. Cada aba é autocontida (estado, filtros, fetch, tabela, paginação):
- `AbaProcessos` — lista principal de processos sem polo passivo.
- `AbaContatos` — contatos extraídos (`contatos_reclamadas`), com export CSV.
- `AbaSnov` — contatos enriquecidos via Snov.io (`contatos_snov`), com export CSV.
- `AbaContratos` — upload + análise de risco (Gemini), bloqueada se não premium (`BloqueioPremium`).
- `AbaConversoes` — upload + OCR (Nanonets), bloqueada se não premium.
- `AbaAnalytics` — dashboard interno com gráficos simples desenhados à mão em SVG (`BarChart` , `DonutChart`) — sem lib de gráficos.
- Cada aba premium usa `isPremium` (vindo de `useAuth()`) para decidir entre mostrar conteúdo ou `<BloqueioPremium>` com CTA para `/upgrade` .
- `app/upgrade/page.js` — página estática de comparação de planos, sem integração de pagamento (não há checkout/gateway no código — upgrade de tier é manual via SQL, ver seção 7).

Se o arquivo `dashboard/page.js` continuar crescendo, vale considerar quebrar cada `Aba*` em seu próprio arquivo (`app/dashboard/abas/AbaX.js`) — hoje isso ainda não foi feito, então ao adicionar uma aba nova, siga o padrão das existentes no mesmo arquivo em vez de introduzir uma estrutura diferente pela metade.

12. Fluxo de dados ponta a ponta

Para debugar "por que esse processo não aparece no dashboard", siga esta cadeia:

1. **Coleta**: `worker/index.js` -> `processarJob()` -> `semAdvogado()` decide se o processo entra. Se o processo tem advogado no polo passivo (ou se a verificação deu erro), ele é descartado silenciosamente. Rode com `DEBUG_HTML=1` para inspecionar o HTML da página de detalhe que o worker está lendo.
2. **Persistência**: `INSERT ... ON DUPLICATE KEY UPDATE` em `processos_sem_polo_passivo`. Confira com `SELECT * FROM execucoes ORDER BY id DESC LIMIT 5;` se a última execução rodou e quantos processos novos ela gravou.
3. **API**: `GET /api/processos` (`processos.controller.ts`) monta a query com os filtros da querystring — confira se um filtro (vara, data, número, reclamada) está descartando a linha esperada.
4. **Frontend**: `AbaProcessos` em `dashboard/page.js` chama `authFetch` com os filtros do formulário — confira o Network tab do browser para ver a query real enviada e a resposta bruta da API.

13. Como adicionar uma feature nova (receita)

Exemplo: endpoint novo de listagem autenticado.

1. Se precisar de tabela nova, crie `database/00N_nome.sql` com `CREATE TABLE IF NOT EXISTS ...` seguindo o estilo das migrations existentes (chaves, índices, `ENGINE=InnoDB DEFAULT CHARSET=utf8mb4`). Rode `node migrate.js` localmente para aplicar.
2. Crie a pasta do módulo em `backend/src/<nome>/` com `<nome>.controller.ts` (e `<nome>.service.ts` se a lógica não for trivial SQL direto).
3. Use `DbService` injetado no construtor para consultas. Se o endpoint for uma listagem, siga o formato de resposta paginada da seção 6.
4. Proteja com `@UseGuards(JwtAuthGuard)` (e `TierGuard` + `@Tiers(...)` se for recurso premium/admin).
5. Registre o novo controller/provider em `backend/src/app.module.ts` (`imports` / `controllers` / `providers`) — sem isso o NestJS não o enxerga.
6. `cd backend && npm run build` para conferir que compila sem erro de tipo.
7. No frontend, adicione a chamada usando `authFetch` de `useAuth()` (importe de `@/lib/auth`), seguindo o padrão de uma aba existente (`AbaContatos` é um bom exemplo simples de listagem com filtros e paginação).
8. Se o recurso é premium, cheque `isPremium` do `useAuth()` no frontend *e* garanta o guard `@Tiers()` no backend — nunca confie só na checagem do frontend, ela é só UX.
9. Teste manual: crie usuário, promova tier via SQL se necessário, teste os dois caminhos (autorizado/bloqueado).

14. Dívida técnica e pontos de atenção conhecidos

Situações reais encontradas no código atual — leia antes de assumir que algo "deveria" funcionar de um jeito e não funciona:

- **Alertas por e-mail parecem desconectados.** `AlertasService. dispararNotificacoes()` existe, monta e envia e-mail para quem tem alerta ativo batendo com processo novo — mas **nada no código chama esse método**. Não é chamado pelo worker, nem por nenhum endpoint/cron. O README descreve isso como parte do fluxo diário ("Sistema verifica alertas premium e envia e-mails automaticamente"), mas hoje isso não acontece sozinho — é preciso disparar manualmente ou reconectar esse método a algum gatilho (ex.: chamar ao final do worker, ou expor endpoint interno chamado pelo cron). Os commits recentes (`fix: remove email alerts`, `fix: remove email alerts2`) sugerem que isso foi desativado de propósito — confirme com o time antes de reativar.
- **Duas tabelas usadas no código não têm migration no repo**: `contatos_snov` (usada por `contatos-snov.controller.ts` e `snov-live.service.ts`) e `sessoes_acesso` (usada por `auth.controller.ts` e `analytics.controller.ts`). Um banco criado do zero só com `database/*.sql` vai faltar essas tabelas. Ao mexer nesses módulos, ou formalize a migration que falta, ou pelo menos documente o `CREATE TABLE` real usado em produção.
- **Caminho hardcoded no worker**: `backend/worker/index.js` linha 12 aponta para `/root/jte-monitor/backend/.env`, enquanto o resto da documentação assume `/opt/jte-monitor`. Confira/ajuste conforme o servidor real antes de rodar via cron em um ambiente novo.
- **Lógica de enriquecimento Snov.io duplicada**: `snov-live.service.ts` (chamada on-demand pela API) e `contatos/snov.js` (script batch standalone) implementam basicamente o mesmo fluxo de chamadas à Snov.io. Mudanças em um provavelmente precisam ser replicadas no outro.

- **Limite do tier free está desligado**: `FREE_LIMIT = 999999` em `processos.controller.ts` — comentário no próprio arquivo explica como reativar (mudar a constante + rebuild). Hoje `free` e `premium` veem a mesma quantidade de processos; a diferença de tier hoje é só alertas/filtros salvos e os módulos de Contratos/Conversões.
- **Sem testes automatizados nem CI configurado** neste repositório — toda validação hoje é manual. Se for introduzir testes, `ts-node-dev`/Jest ainda não estão configurados no `backend/package.json`.
- **Sem service layer em todo lugar** (ver seção 6) — é intencional/estilo do projeto, não um bug, mas pode surpreender quem espera um padrão NestJS "canônico" com repositórios.
- **Não há gateway de pagamento** — `/upgrade` é só uma vitrine estática; upgrade de tier é manual via SQL (seção 7). Se for integrar um checkout, não existe nenhuma base de billing no código para reaproveitar.

15. Onde procurar cada coisa (índice rápido)

Preciso mexer em...	Arquivo
Regras de quem pode logar / tokens	`backend/src/auth/auth.service.ts`
Cookies de refresh token, endpoints de auth	`backend/src/auth/auth.controller.ts`
Quem pode acessar recurso premium	`backend/src/common/guards/index.ts`
Pool de conexão MySQL	`backend/src/common/db.service.ts`
Filtros/paginação da lista principal de processos	`backend/src/processos/processos.controller.ts`
Envio de e-mail de alerta	`backend/src/alertas/alertas.controller.ts` (classe `AlertasService`)
Enriquecimento Snov.io on-demand	`backend/src/contatos/snov-live.service.ts`
Enriquecimento Snov.io em lote	`contatos/snov.js`
Análise de contrato (IA)	`backend/src/contratos/gemini.service.ts`
OCR de documento	`backend/src/conversoes/nanonets.service.ts`
Analytics interno (admin)	`backend/src/admin/analytics.controller.ts`
Registrar controller/provider novo	`backend/src/app.module.ts`
Lógica de scraping do JTe	`backend/worker/index.js`
Criar/alterar tabela	`database/*.sql` + `migrate.js`
Login/registro/estado de sessão (frontend)	`frontend/src/lib/auth.js`
Qualquer aba do dashboard	`frontend/src/app/dashboard/page.js`

Preciso mexer em...	Arquivo
Comparação de planos	`frontend/src/app/upgrade/page.js`
Variáveis de ambiente	`backend/.env.example`, `frontend/.env.local.example`, seção 13 do `README.md`
Deploy, systemd, Nginx, HTTPS, cron	`README.md` (seções 6–10)